Laboratorio 7 — Autenticación

Profesor: Matías Toro
Ayudantes: Benjamín Indo, Benjamín Madrid, Camilo Huerta, Daniel Ramírez, Julián Ferreira, Sebastián Bustos, Sebastián González, Tomás Méndez, Tomás Ubilla

En este laboratorio pondrán en práctica los conceptos vistos en la clase de *Autenticación*. Encontrarán la template para este laboratorio en la sección de material docente en U-Cursos.

El objetivo es agregar usuarios y *login* con JWT a la aplicación de threads, manteniendo la posibilidad de navegar y comentar como invitado. La template trae el *backend* organizado como en clases, con `cookie-parser` configurado y los archivos por completar marcados con `TODO (Pn)`, y el *frontend* con el *proxy* de Vite hacia `/api` y la página de registro lista. Para probar los *endpoints* puede usar `backend/requests.rest` con la extensión *REST Client* de VS Code.

P1.

Complete el esquema de usuarios en `models/user.ts` con los campos `username`, `email` y `passwordHash`, todos obligatorios. No pueden repetirse ni el `username` ni el `email`, y el correo debe validarse con la expresión regular `/^[^\s@]+@[^\s@]+\.[^\s@]+$/`. Al transformar un usuario a JSON no deben aparecer `_id`, `__v` ni `passwordHash`.

Agregue además al esquema de publicaciones un campo `user` que referencie a su autor. Las publicaciones anónimas no lo tendrán.

P2.

Implemente `POST /api/users`, que recibe `username`, `email` y `password` y guarda el usuario con el *hash* de la contraseña (use `bcrypt`), y `GET /api/users`, que lista los usuarios. Responda `400` con un mensaje de error si la contraseña tiene menos de 3 caracteres, si el correo es inválido, o si el `username` o el `email` ya existen, indicando cuál de los dos.

P3.

Implemente `POST /api/login`. Si las credenciales son incorrectas, responda `401` sin revelar si falló el usuario o la contraseña. Si son correctas, genere un JWT con el `id`, el `username` y un token CSRF (`crypto.randomUUID()`), que expire en 1 hora. Envíe el JWT en una *cookie* `httpOnly` llamada `token`, el token CSRF en el *header* `X-CSRF-Token`, y el `username` en el cuerpo.

P4.

Implemente el *middleware* `withUser`, que verifica el JWT de la *cookie* `token`, compara su campo `csrf` con el *header* `X-CSRF-Token` y guarda el `id` en `request.userId`, o responde `401` si algo falla. Úselo en `GET /api/login/me`, que responde con el usuario de la sesión, e implemente `POST /api/login/logout`, que borra la *cookie*. Agregue al `errorHandler` el caso `TokenExpiredError` con un `401`.

P5.

Cree un *middleware* `withOptionalUser` a partir de `withUser`, que deje pasar las peticiones sin *cookie*, y úselo en la creación de threads y comentarios (`POST /api/threads` y `POST /api/threads/:id`). Con

sesión, el autor es el `username` y se guarda el campo `user`, ignorando el `author` del cuerpo. Sin sesión, el autor es el `author` del cuerpo o `Anónimo` si no viene. Si la *cookie* trae un token inválido, responda `401`.

P6.

Conecte el *frontend* con la autenticación:

- En `services/login.ts`, implemente `login` (guarda el token CSRF del *header* en `localStorage`), `restoreLogin` (consulta `/api/login/me`) y `logout` (borra el token).
- En `utils/axiosSecure.ts`, cree una instancia de `axios` que envíe el *header* `X-CSRF-Token` en cada petición, y úsela para crear threads y comentarios.
- Complete `pages/Login.tsx` y el botón para cerrar sesión en `components/TopBar.tsx`, y restaure la sesión al montar `App.tsx`.